NLP_ASSIGNMENT

Longitudinal Clinical Condition Extraction Pipeline

Submission Report

Divyansh Jaiswal
Mechanical (4th Year)
IIT (BHU) Varanasi

Content

Content **2**
1. Executive Summary **3**
2. Problem Challenges **4**
3. Final Pipeline Architecture **4**
Stage 1 — Input Loading (loader.py) 4
Stage 2 — Section Parsing (parser.py) 5
Stage 3 — Patient-Level LLM Extraction (llm_extractor.py) 5
Stage 4 — LLM Normalization (normalizer.py & llm_normalizer.py) 5
Stage 5 — Aggregation (aggregator.py & post_aggregator.py) 6
Stage 7 — Taxonomy Mapping (mapper.py) 6
Stage 8 — Validation and Output (validator.py) 6
4. Design Decisions **6**
4.1 Why Hybrid Instead of Pure LLM? 6
4.2 Why Single Patient-Level Prompt? 7
4.3 Why Multi-Stage Post Processing? 7
4.4 Why Precision Focus? 8
5. Experiments and Iterations **8**
5.1 Prompt Iteration 8
5.2 Model Comparison 8
GPT-OSS 9
Gemini-Compatible Endpoint 9
5.3 Mapper Strictness 9
6. What Worked Well **9**
Strong Components 9
7. What Did Not Work Well **10**
8. Ideas Explored but Not Fully Integrated **10**
9. Efficiency Considerations **12**
Key Efficiency Choices 12
Overall Goal 13
10. How to Run **13**
11. Conclusion **13**

1. Executive Summary

The task is to build a system that extracts a **structured condition summary** for each patient from their clinical notes.

For each Patient, produce a JSON file containing every condition found across all their notes, with the following fields per conditions:

```json
{
  "patient_id": "patient_XX",
  "conditions": [
    {
      "condition_name": "Descriptive name of the condition",
      "category": "category_key",
      "subcategory": "subcategory_key",
      "status": "status_key",
      "onset": "March 2014",
      "evidence": [
        {
          "note_id": "text_0",
          "line_no": 12,
          "span": "The exact text from the note supporting this condition"
        }
      ]
    }
  ]
}
```

Clinical notes are noisy, repetitive, partially structured, and distributed across multiple encounters. The same condition may appear in different wording across notes, and some mentions are historical while others are active.

To solve this, I developed a **hybrid pipeline** combining deterministic preprocessing with Large Language Model (LLM)-based extraction and normalization.

The final system uses:

- Note loading and chronological processing
- section-aware parsing
- patient-level LLM extraction
- normalization
- duplicate aggregation

- taxonomy-constrained Batch mapping
- schema validation

The design prioritizes:
- precision (false positive control)
- reproducibility
- low API call count
- scalable patient-level processing
- practical engineering robustness

2. Problem Challenges

This task contains several real-world NLP challenges:
1. **Longitudinal Notes:** patient may have multiple notes over time in which conditions appear repeatedly, change of Status, Historical mentions.
2. **Clinical Synonyms:** The same disease may appear in different form (Arterial hypertension, HTN, Hypertension)
3. Notes contain some sections having **structured lists**, while others have a **narrative prose**.
4. **False Positives:** Symptoms, procedures, devices, and vague findings can be incorrectly extracted as diseases.
5. **Taxonomy Mapping:** Conditions must be assigned to valid categories/subcategories.

3. Final Pipeline Architecture

The implemented runtime pipeline follows the stages below.

Stage 1 — Input Loading (loader.py)

The pipeline reads:
- **--data-dir**
- **--patient-list**
- **--output-dir**

Each patient's note files are loaded and processed independently.

Dates are parsed when available to preserve chronology.

Stage 2 — Section Parsing (parser.py)

Each note is segmented into clinically meaningful sections such as:

- Diagnoses
- Other Diagnoses
- Medical History
- Imaging
- Histology
- Lab Results
- Physical Examinations
- Current Presentation
- Therapy and Progressions

This improves signal quality by preserving section context.

Stage 3 — Patient-Level LLM Extraction (llm_extractor.py)

All parsed note text for a patient is combined into a single structured prompt.

The LLM extracts:

- **condition_name**
- **category**
- **status**
- **source_id**

This patient-level batching strategy was intentionally chosen instead of many note-level calls.

Advantages

- One API call for a patient
- lower cost
- lower latency
- better cross-note context

Stage 4 — LLM Normalization (normalizer.py & llm_normalizer.py)

Extracted mentions are canonicalized into consistent medical names.

Examples:

- Arterial hypertension → hypertension
- Non-insulin-dependent diabetes mellitus → type 2 diabetes mellitus
- Oral thrush → oral candidiasis

Normalization also corrects misclassified categories when needed.

Stage 5 — Aggregation (aggregator.py & post_aggregator.py)

Repeated mentions across multiple notes are merged, to present the evidence across notes for the conditions that are normalized. Based on all the evidence listed across notes, the **status**(based on the latest note) and **onset**(based on the earliest note) is decided.

Example: Three mentions of hypertension across notes become one final condition, with an evidence list of all three places.

Aggregation preserves evidence spans while reducing duplicates.

Stage 7 — Taxonomy Mapping (mapper.py)

Each condition is mapped into the official task taxonomy. The mapper assigns subcategory using LLM. Here category wise batching is done, and is passed to LLM. This was done to reduce, the LLM API calls.

After running mapper, it's output is formatted according to JSON output needed.

Low-confidence or invalid mappings are removed. This was helpful in reducing false positives.

Stage 8 — Validation and Output (validator.py)

Each patient output is validated against the required JSON schema and saved as required.

4. Design Decisions

4.1 Why Hybrid Instead of Pure LLM?

A fully prompt-only pipeline was not ideal for this task because clinical extraction requires both semantic reasoning and structural consistency.

Pure LLM systems often showed:

- unstable outputs across runs
- duplicate conditions
- inconsistent naming
- noisy taxonomy assignments
- occasional unsupported predictions

To address this, I designed a **hybrid architecture** where LLM components handle language understanding, while deterministic components enforce structure and consistency.

LLM Responsibilities

- condition extraction from free text
- synonym normalization
- taxonomy mapping support

Deterministic Responsibilities

- note loading and chronology
- section parsing
- duplicate aggregation
- schema validation
- output formatting
- confidence filtering

This combination provided stronger reliability than either approach alone.

4.2 Why Single Patient-Level Prompt?

Initial experiments considered separate LLM calls per note or per section.

The final design uses one patient-level extraction prompt containing all parsed notes.

Benefits

- One API request per patient, for extraction.
- lower token overhead from repeated instructions
- better cross-note context
- improved recognition of repeated diseases
- easier deployment and batching

This was especially useful for longitudinal reasoning.

4.3 Why Multi-Stage Post Processing?

Raw extraction output is not sufficient for final structured submission.

Therefore extracted conditions pass through multiple refinement stages:

LLM-assisted Refinement

- normalization of synonyms
- category correction
- subcategory mapping

Deterministic Refinement

- duplicate merging
- evidence consolidation
- schema enforcement
- invalid output removal

This converts raw noisy mentions into cleaner patient-level structured outputs.

4.4 Why Precision Focus?

In medical extraction tasks, false positives are expensive because they create incorrect patient conditions.

Typical low-value false positives include:

- symptoms only
- procedures
- anatomical terms
- vague radiology findings
- weakly implied diagnoses

Later iterations intentionally favored precision through stricter prompts, normalization consistency, and taxonomy filtering.

5. Experiments and Iterations

A major focus of the project was systematic experimentation.

5.1 Prompt Iteration

Multiple extractor prompts were tested.

Broad Prompts

Pros: higher recall
Cons: many false positives

Precision Prompts

Pros: cleaner outputs, fewer hallucinations
Cons: slightly lower recall
Final system used a balanced prompt. You can see in **llm_extractor.py** file.

5.2 Model Comparison

Two OpenAI-compatible backends were tested.

GPT-OSS

Observed:

- stronger precision
- better normalization consistency
- fewer noisy extractions

Gemini-Compatible Endpoint

Observed:

- competitive recall
- occasionally noisier outputs
- some variability under rate limits

5.3 Mapper Strictness

Two strategies were tested:

Force nearest subcategory

Higher recall, more wrong mappings.

Return unknown if uncertain

Lower false positives, but increased false negatives.
Final version used stricter mapping logic.

6. What Worked Well

Strong Components

Section-Aware Extraction

Preserving note sections such as Diagnoses, Medical History, Imaging, and Lab Results improved extraction relevance and reduced noise from unstructured text.

Condition Normalization

LLM-based normalization reduced synonym fragmentation and standardized repeated mentions into cleaner canonical condition names.

Examples:

- Arterial hypertension → hypertension
- NIDDM → type 2 diabetes mellitus

Aggregation

Repeated mentions across multiple notes were merged successfully, producing cleaner patient-level outputs.

Category-Wise Batched Mapping

Grouping conditions by category before mapper calls reduced API overhead and improved subcategory consistency.

Patient-Level Extraction Calls

Using one extraction call per patient was efficient and preserved cross-note context.

7. What Did Not Work Well

False Negatives from Strict Filtering

Aggressive mapper filtering sometimes removed valid but uncommon conditions, increasing false negatives.

Rare Disease Mapping

Some uncommon or highly specific diagnoses remained difficult to assign confidently to subcategories.

Status Classification

Differentiating active vs resolved historical conditions remained challenging when wording was ambiguous.

Long Prompt Instability

Very large patient-level prompts occasionally produced inconsistent extraction behavior across runs.

Precision–Recall Tradeoff

Prompt settings that reduced false positives sometimes also reduced recall.

8. Ideas Explored but Not Fully Integrated

During development, I explored several practical extensions intended to improve precision, reduce LLM usage, and make the pipeline more cost-efficient. Due to time constraints, these ideas were not fully integrated into the final submitted runtime pipeline.

Rule-Based Lab Condition Extraction

Certain laboratory abnormalities can be converted directly into clinical conditions using deterministic thresholds.

Examples:

- low hemoglobin → anemia
- elevated INR or low Quick value → coagulopathy
- low lymphocyte count → lymphopenia

This approach could improve recall while avoiding additional LLM calls for straightforward laboratory-derived conditions.

Additional Rule-Based Post Processing

I also explored lightweight deterministic post-processing after aggregation, such as:

- singular/plural normalization
- common synonym merging
- removal of vague non-diagnostic findings
- removal of symptom-only mentions
- duplicate cleanup across repeated notes

These rules can improve final precision and reduce noise before later stages.

Confidence-Based Routing Strategy

A more advanced extension considered was a confidence-based multi-stage pipeline.

After extraction, predicted conditions could be assigned confidence scores and separated into buckets.

High Confidence Bucket

Very confident conditions could bypass expensive downstream LLM refinement and use:

- direct cache lookup
- deterministic normalization
- direct taxonomy mapping

Medium / Low Confidence Bucket

Uncertain conditions could be routed to an additional LLM refinement stage using previously cached normalized conditions as context.

This would focus expensive reasoning only on difficult cases.

Potential Benefits

- lower token cost
- faster runtime
- better scalability
- fewer unnecessary API calls
- more efficient inference

Taxonomy Shortcuts for Common Conditions

For frequent high-confidence diseases, deterministic direct mapping to subcategories could replace some mapper LLM calls.

Examples:

- hypertension
- coronary artery disease
- hypothyroidism
- type 2 diabetes mellitus

This could preserve accuracy while lowering cost.

9. Efficiency Considerations

The evaluation also considers runtime speed and total token usage, so efficiency was an important design priority.

Key Efficiency Choices

- **Single extraction call per patient**
  All parsed notes for one patient were combined into one extractor prompt instead of making separate calls per note.
- **Single normalization call per patient**
  Extracted conditions for a patient were normalized together in one LLM call instead of separate calls for each condition.
- **Aggregation before mapping**
  Duplicate and repeated conditions were merged before mapper stage, reducing the number of conditions requiring subcategory assignment.
- **Category-wise batched mapping**
  Conditions from the same category were grouped together for mapper calls, reducing repeated taxonomy prompts.
- **Deterministic preprocessing**
  Lightweight non-LLM logic was used for:
  - note loading
  - section parsing
  - duplicate merging
  - formatting
  - schema validation
- **Reduced downstream workload through cleaner extraction**
  Better extraction precision reduced unnecessary normalization and mapping of false positives.
- **Patient-wise modular processing**
  Each patient was processed independently, making the pipeline easier to scale and debug.

Overall Goal

Use LLM reasoning only where it adds value, while minimizing redundant calls, token usage, and runtime.

10. How to Run

setx OPENAI_BASE_URL "https://api.example.com/v1"
setx OPENAI_API_KEY "your_key"
setx OPENAI_MODEL "model_name"
Setx USE_GEMINI “False” // Make it True When want to use GEMINI model with its API KEY

setx GEMINI_API_KEY “gemini_key”
I used this because of higher rate limit access I got for my experimentations.

After doing this in Terminal, kill the terminal, and open it, then run below, code

```
python main.py \
  --data-dir ./data/dev \
  --patient-list ./patients.json \
  --output-dir ./output
```

(OPTIONAL, should run before main file run)
$env:OPENAI_MODEL
$env:OPENAI_API_KEY
$env:OPENAI_BASE_URL

11. Conclusion

This project demonstrates that a carefully engineered hybrid system can effectively solve structured longitudinal clinical extraction.

Rather than relying only on prompting, the final pipeline combines:

- LLM reasoning
- deterministic structure
- normalization
- aggregation
- taxonomy constraints

This produced a practical, reproducible, and cost-aware solution suitable for evaluation on unseen patient data.